or playing a game, then you would want graphical processing units (GPUs) in your system that could handle the render with ease.

That is what GPU-enabled hardware acceleration is when it comes to simulation software and games. In the context of robotics, hardware acceleration can help you create faster and more power-efficient robots. This is done through various accelerator platforms.

Let us now understand how hardware acceleration is used in robotics. We all have seen the Atlas robot by Boston dynamics. The Atlas uses hardware acceleration in the perception stack to navigate through the obstacles. It uses a time of flight sensor at high frequency to detect and extract surfaces from the environment and then use the navigation stack to navigate and control stack to move through the environment.

It is important to know that all of this has to happen at the edge as you cannot have these computations in the cloud, because the environment is dynamic and constantly changing. The obstacle here is that you cannot afford the latency that cloud computations would have.

This is where the edge devices and special hardware acceleration come in because edge devices by themselves are not fast enough to deal with this situation. That is where you put the hardware accelerator into the mix and get optimal performance.

Let us first look at the compute architectures and how they are used in the robotics context. We can do this by taking the analogy of a factory. A CPU can be thought of as a very generalised workshop where you have very general tools available and you can create anything you want from it. The only bottleneck is that any sort of computation happens sequentially and not parallelly. You can build anything but it just takes some time because things happen sequentially.

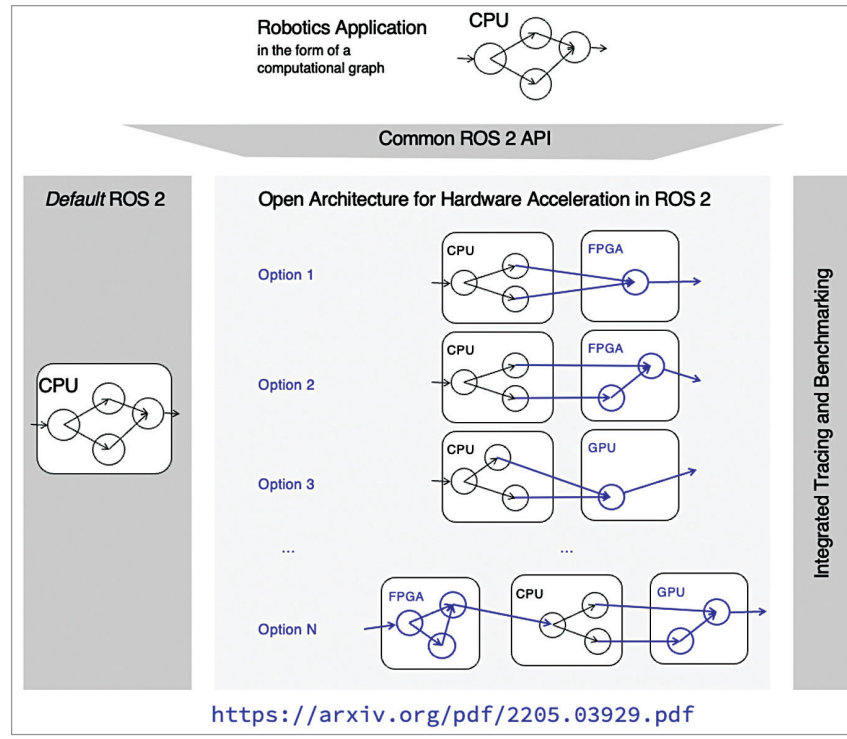

Fig. 1: Hardware acceleration framework for ROS

Then we have the GPUs, which are basically a very streamlined form of a workshop where there are a lot of workers available but the workers can do only certain tasks. They have a very limited number of tools available to them. GPUs can perform tasks which have a lot of data to deal with parallelly.

From a software engineering perspective, the CPU deals with data points while the GPU deals with data vectors. Therefore, they can parallelly process a lot of computations together. Now, the problem with this is that GPUs require a hardware system expert to develop the architecture to decide what sort of tools are available to an individual worker. That drives up the cost of a GPU. Also, it is not power efficient.

Field programmable gate arrays (FPGAs) are basically factories which can be transformed to do a specific task. Through programs like OpenCL, you can re-configure your factory and produce a specific factory for the product you want to design. The benefit of FPGAs is that they consume low power and give high performance. If you look at the current literature in robotics perception then you will see that FPGAs outperform GPUs in many tasks.

We also have application-specific integrated circuits (ASICs), which are very powerful as well as performance efficient. The problem is that they require high level of hardware expertise and the architectures that are available for ASIC are few

Production-grade multi-platform ROS support with Yocto

Instead of relying on common development-oriented Linux distros (such as Ubuntu), our contributions to Yocto allow to build a customised Linux system for your use case with ROS, providing unmatched granularity, performance, and security.